EE 789
Assignment 2: Matrix Multiplication

April 6, 2026

You are expected to design a circuit which multiplies two square matrices. In particular the circuit works on two 64x64 matrices A and B with entries being $\$int < 8 >$ values.

If $A = [a_{ij}]$ and $B = [b_{ij}]$ $(0 \leq i, j \leq N-1)$ then the product matrix $C = A \times B$ has entries c_{ij} with

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj}$$

A matrix can be viewed in row-form or column form. In row-form, we can write

$$A = \begin{pmatrix} rA_0^T \\ rA_1^T \\ \dots \\ rA_{N-1}^T \end{pmatrix}$$

where rA_0^T is the first row and so on. In column form, we can write A as

$$A = (cA_0 \; cA_1 \; \dots \; cA_{N-1})$$

where cA_0^T is the first column and so on. Similarly for B.

The product matrix C can be built in different ways.

$$c_{ij} = rA_i^T . cB_j^T$$

or (as a sum of rank-1 matrices):

$$C = \sum_{i=0}^{N-1} cA_i X rB_j^T$$

1 The naive algorithm

A is stored in row major form and B is stored in row major form.

The base algorithm for matrix multiplication in which A, B are two 8×8 matrices each stored in row major will be circulated to you. A test-bench is provided. Confirm that

- You have understood the code.
- The code compiles (make) and the simulation runs correctly. Observe that the number of clock cycles requred to complete the matrix multiplication is approximately 44K. By extrapolation, multiplication of a 64x64 matrix in this manner will require 44K X 512 clock cycles!

2 Optimization 1 (loop-unrolling): loop unroll in the dot product

The basic loop in the dot-product is:

```
$parameter N 8
$storage A B : $array[N][N] $of $uint<8>
$module [dotp] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp] {
         $dopipeline $depth 7 $fullrate

         $merge $entry $loopback
              $phi SUM := ($bitcast ($int<8>) 0)
                                  $on $entry nSUM $on $loopback
              $phi K   := $zero<8> $on $entry nK $on $loopback
         $endmerge

         $volatile nK := (K + 1)
         $volatile K_continue := (K < {N-1})

         nSUM := (SUM + (A[I][K] * B[K][J]))
         $while K_continue
      } (nSUM => nSUM)
```

```
      R := nSUM
}
```

Unroll the loop:

```
$module [dotp_unroll_2] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp] {
         $dopipeline $depth 7 $fullrate

         $merge $entry $loopback
              $phi SUM0 := ($bitcast ($int<8>) 0)
                              $on $entry nSUM0 $on $loopback
              $phi SUM1 := ($bitcast ($int<8>) 0)
                              $on $entry nSUM1 $on $loopback
              $phi K    := $zero<8> $on $entry nK $on $loopback
         $endmerge

         $volatile nK := (K + 2)
         $volatile K_continue := (K < {N-2})

         nSUM0 := (SUM0 + (A[I][K] * B[K][J]))
         nSUM1 := (SUM1 + (A[I][(K + 1)] * B[(K+1)][J]))

         $while K_continue
     } (nSUM0 => S0 nSUM1 => S1)

     R := (S0 + S1)
}
```

Unrolling is a useful way to hide the accumulator loop dependency delay.

3 Optimization 2 (word-parallelism): read many elements of the matrices at a time.

Let N=64. The matrix A is in row major form and the matrix B is in column major form.

Fetch 8 elements (ie 64-bits) of a row in matrix A and and column in matrix B at a time, and multiply the two 8-element vectors using 8 multipliers and 7 adders.

Here is how you can do it.

```
$parameter N 64
// The array elements are 64-bits wide..
$storage A B : $array[{N}][{N/8}] $of $uint<{8*8}>
$module [dotp] $in (I J: $uint<8>) $out (R: $int<8>) $is
{
      $branchblock[dotp] {
         $dopipeline $depth 7 $fullrate

         $merge $entry $loopback
              $phi SUM := ($bitcast ($int<8>) 0)
                                  $on $entry nSUM $on $loopback
              $phi K   := $zero<8> $on $entry nK $on $loopback
         $endmerge

         $volatile nK := (K + 1)
         $volatile K_continue := (K < {{N/8}-1})
         nSUM := (SUM + ($call dot8 (A[I][K], B[J][K])))

         $while K_continue
      } (nSUM => nSUM)
      R := nSUM
}

$pipeline $depth 7 $fullrate $operator
      $module[dot8] $in (A B: $uint<{8*8}) $out (R: $int<8>)
      $is
{
  $volatile $split (A 8 8 8 8 8 8 8 8)
      (A0 A1 A2 A3 A4 A5 A6 A7)
  $volatile $split (B 8 8 8 8 8 8 8 8)
      (B0 B1 B2 B3 B4 B5 B6 B7)

  // type conversions.
```

```
  $volatile iA0 := ($bitcast ($int<8>) A0)
  $volatile iA1 := ($bitcast ($int<8>) A1)
  $volatile iA2 := ($bitcast ($int<8>) A2)
  $volatile iA3 := ($bitcast ($int<8>) A3)
  $volatile iA4 := ($bitcast ($int<8>) A4)
  $volatile iA5 := ($bitcast ($int<8>) A5)
  $volatile iA6 := ($bitcast ($int<8>) A6)
  $volatile iA7 := ($bitcast ($int<8>) A7)

  // type conversions.
  $volatile iB0 := ($bitcast ($int<8>) B0)
  $volatile iB1 := ($bitcast ($int<8>) B1)
  $volatile iB2 := ($bitcast ($int<8>) B2)
  $volatile iB3 := ($bitcast ($int<8>) B3)
  $volatile iB4 := ($bitcast ($int<8>) B4)
  $volatile iB5 := ($bitcast ($int<8>) B5)
  $volatile iB6 := ($bitcast ($int<8>) B6)
  $volatile iB7 := ($bitcast ($int<8>) B7)

   P0 := (iA0 * iB0)
   P1 := (iA1 * iB1)
   P2 := (iA2 * iB2)
   P3 := (iA3 * iB3)
   P4 := (iA4 * iB4)
   P5 := (iA5 * iB5)
   P6 := (iA6 * iB6)
   P7 := (iA7 * iB7)

   S0 := (P0 + P1)
   S1 := (P2 + P3)
   S2 := (P4 + P5)
   S3 := (P6 + P7)

   T0 := (S0 + S1)
   T1 := (S2 + S3)

   R := (T0 + T1)
}
```

4 Optimization 3 (blocking and tiling): divide A, B into blocks.

Let N=64. Divide A, B into 32x32 blocks, do block matrix multiplication and then block result summations to get the best performance you can.

You can use separate memories to keep groups of blocks so that the memory access bandwidth can be increased.

5 Assignment

1. Confirm that the base implementation shared with you works correctly (5 marks).

2. For each optimization shown above, incorporate the optimization in your algorithm and report the improvement in performance (if any!) (15 marks).

3. Use all three optimizations: loop-unrolling, word-parallelism, and blocking and measure the improvement in performance relative to the baseline (10 marks).

6 Submission

Submission will be done in stages:

- Problem 1: by Sunday 5-April 2355.
- Problem 2: by Sunday 12-April 2355.
- Problem 3: by Sunday 25-April 2355.

Late submissions will be given a 2 mark penalty, (at each stage).